RoToRBot - Autonomous Spherical Robot

Techfest 2026-27 · Ultraviolette Wrecking Ball Challenge · Phase 1: CAD & System Document

Team: EigenThinker , Team ID: [**TM-CB1C8E0CEAA**] , Oct 20, 2026 @SAMEER SAAWAN

Abstract

RoToRBot is a 640 mm (2.10 ft), 14.0 kg spherical robot that runs the 75 m track on its own, steering by camera and shifting its centre of gravity to drive and turn.

Two 24 V gearmotors on an internal pendulum turn the polycarbonate shell through 3:1 belts. A 4.9 kg (APPROXIMATE) battery-and-steel sled slides ±70 mm along the axle to tilt the ball and steer it. The centre of gravity sits 116 mm below the sphere centre, so the robot climbs a 15° ramp with a 49° pendulum swing and holds up to 18° inside its 60° swing limit (as per mathematical model) .

The two cameras ride on the ends of a non-rotating axle, 83 mm outside the shell, so they never spin with the ball. A servo inside the shell keeps that axle level while the pendulum swings. The left camera follows the yellow cones, the right camera the red cones, and their overlap gives stereo range on a 0.73 m baseline.

A Raspberry Pi 5 runs vision and an ESP32-S3 runs the 1 kHz drive, pendulum-damping and steering loops. An independent 2.4 GHz kill receiver opens the relay that feeds every actuator, and opens it on its own if the signal is lost. Design target: a clean run in under 90 s with no cone contact.

SEPARATE INNOVATION : The power system is also the test bed for our own long-term concept: a Kandó-inspired switched-capacitor network that lets the robot run well on salvaged, mismatched cells. It stays optional until bench tests prove it.

Compliance with the problem statement

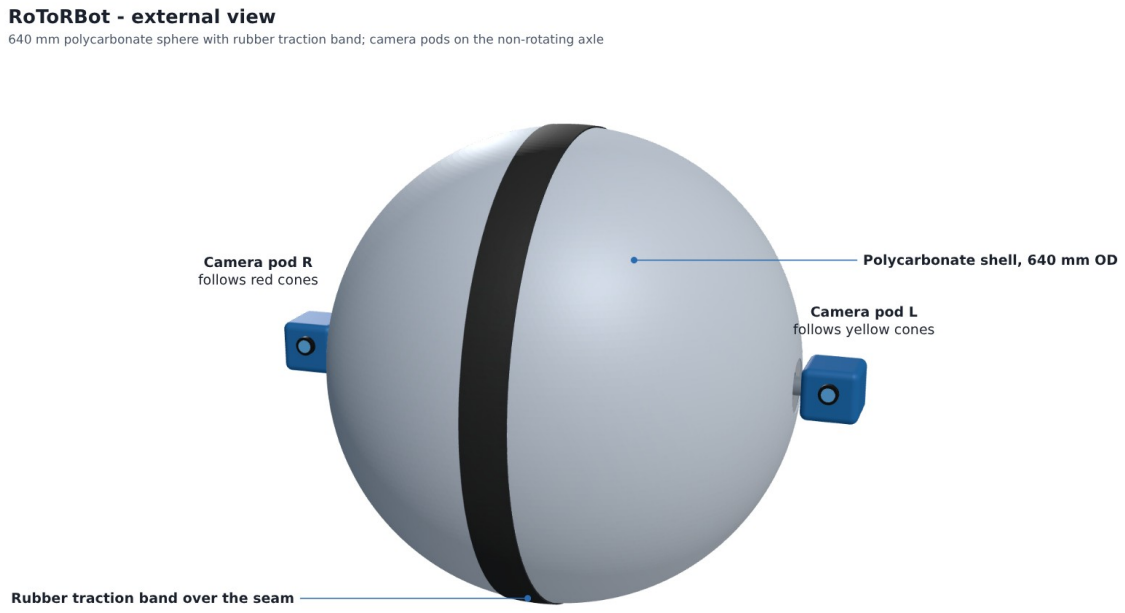
RoToRBot meets every hardware and run rule in the problem statement, with 30 mm of margin on the minimum diameter and 70 mm on the peripheral limit.

Rule	Requirement	RoToRBot	Margin
Shape	Spherical, ball-shaped	Full sphere: two bonded polycarbonate hemispheres, flush seam	…..
Diameter	2.0 - 2.5 ft (610 - 762 mm)	640 mm shell; 644 mm over the traction band (2.10 - 2.11 ft)	30 mm over the minimum, 118 mm under the maximum

Rule	Requirement	RoToRBot	Margin
Internal housing	All computing, power, actuators and electronics inside	Pi 5, ESP32-S3, battery, all motors and servos, drivers and relays are inside	Pods hold only cameras, lens windows and passive sockets
Peripheral limit	≤ 0.5 ft (152 mm) beyond the shell	Camera pods end 83 mm (0.27 ft) beyond the shell	70 mm spare
Power	Onboard battery only	CUSTOM Li-ion pack, 21.6 V, 6 Ah, on the steering sled	No tether or external feed
Primary sensor	Onboard cameras for path tracking and gates	Two forward cameras; IMU and encoders only for stabilisation and odometry	—
Autonomy	100 % autonomous, no offloading	All processing onboard; the radio link carries only the kill signal	—
Kill switch	Wireless, stops all motor actuation	Independent receiver opens relay K2, which feeds every actuator; fails safe on signal loss	Demonstrated at Day 1 inspection

Mechanical architecture and CAD assembly

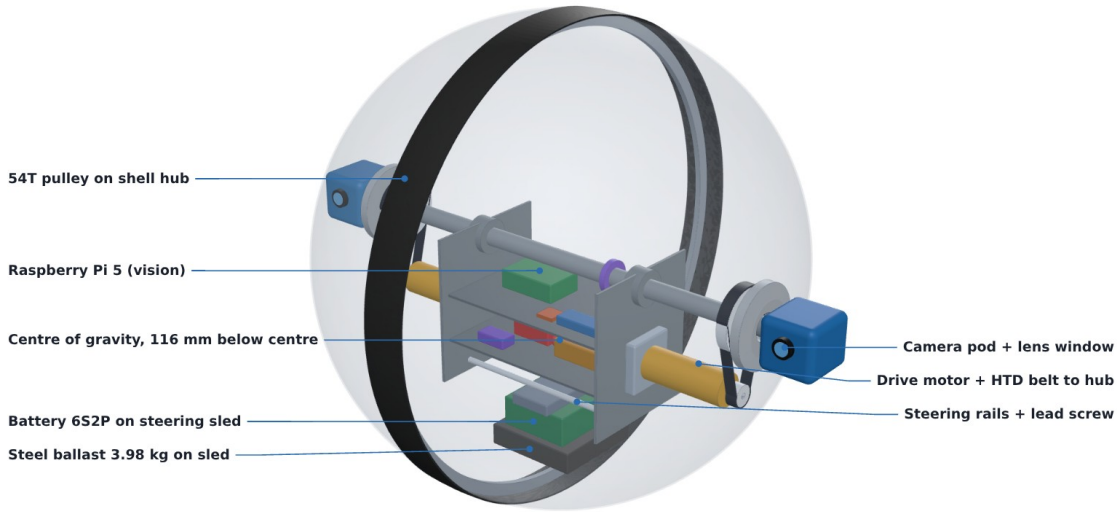
RoToRBot is a pendulum-driven sphere: the shell rolls around a fixed axle, and everything heavy hangs below that axle and never turns with the shell. The full assembly is supplied as RoToRBot_assembly.step (49 named, coloured parts).

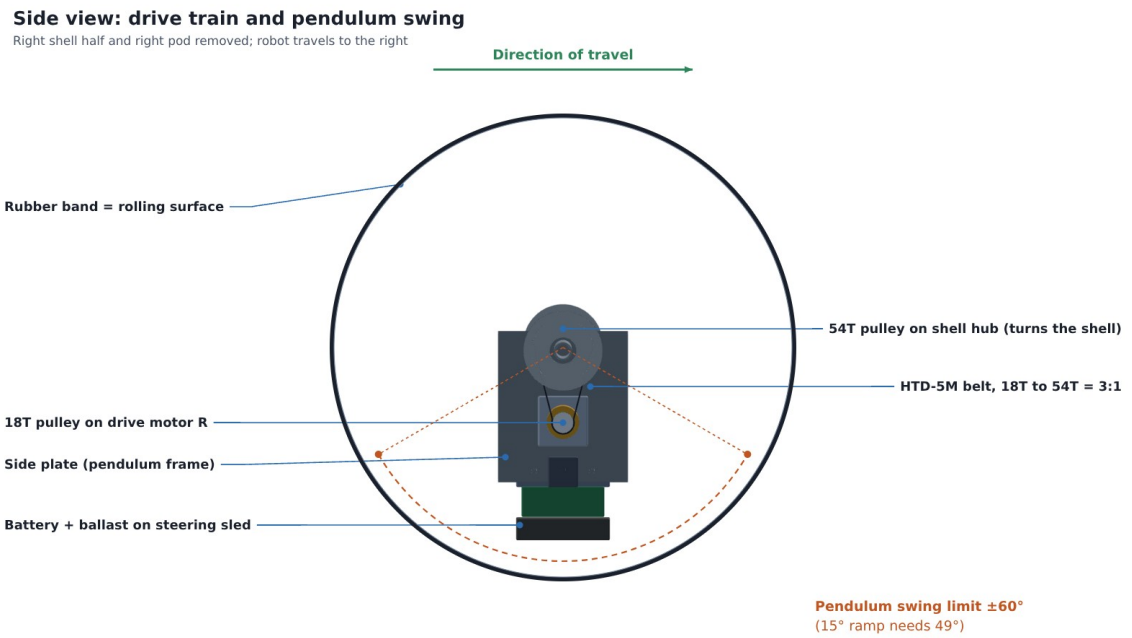


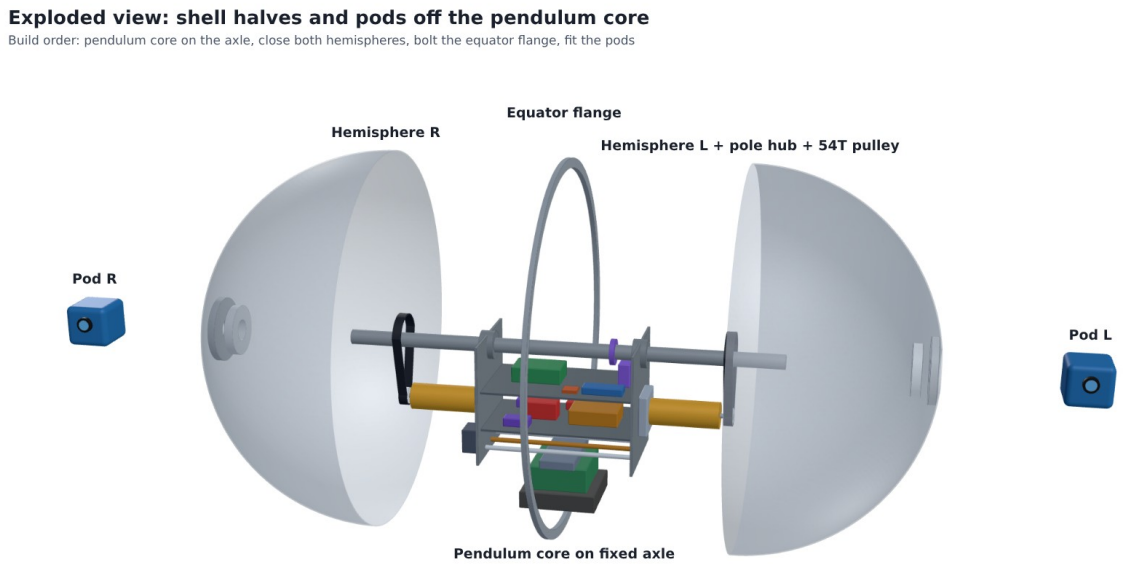
- **Shell (rotates):** two thermoformed polycarbonate hemispheres, 640 mm OD, 2 mm average wall, bolted at an internal equator flange so the outside stays flush. A 60 mm rubber band over the seam is the rolling surface and the grip on the ramps.
- **Fixed axle:** a 25 mm aluminium tube running through 6805 bearings in the two pole hubs. It does not turn with the shell, and the camera cables run inside it, so no slip ring is needed.
- **Camera pods:** one on each axle end, 83 mm outside the shell. Each holds only a camera, a lens window and a passive socket (charging on one side, the arming plug on the other).
- **Pendulum core:** two 4 mm aluminium side plates hang from the axle on bearings. They carry the two electronics decks, both drive motors and the steering sled.
- **Drive:** each 24 V planetary gearmotor turns a 54-tooth pulley on its pole hub through an 18-tooth pulley and HTD-5M belt (3:1). The motor reaction swings the pendulum forward, and that offset weight is what rolls the ball.
- **Steering:** a NEMA 17 stepper and T8 lead screw slide the battery and steel ballast (4.9 kg together) ±70 mm along the axle. The shifted mass tilts the ball up to 12°, and it curves toward that side.
- **Camera levelling:** a servo on the pendulum turns the axle relative to the pendulum, so both pods stay level while the pendulum swings up to ±60°.

RoToRBot - internal assembly, shell shown transparent

Pendulum core hangs from the fixed axle; red sphere = computed centre of gravity





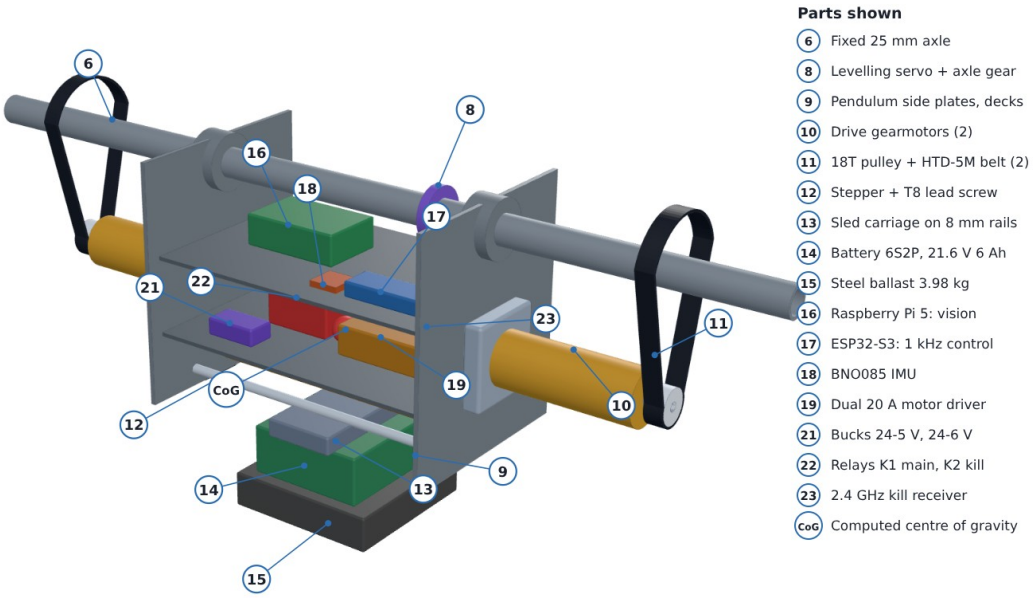


Internal electronic layout

The electronics sit on two decks between the side plates, directly under the axle. The battery and steel ballast hang lowest, on the steering sled.

Internal electronic layout (pendulum core; shell, hubs and pods hidden)

Compute deck A under the axle, power deck B below it, battery and ballast lowest. Numbers match the parts list.

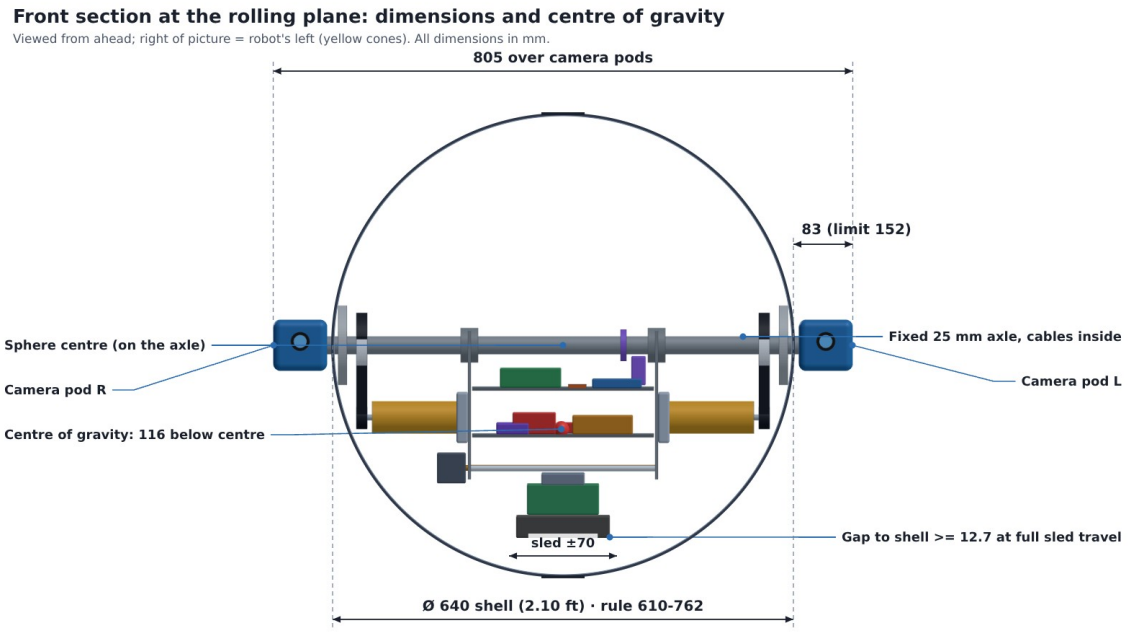


Level (below sphere centre)	What sits there	Why it is there
Deck A, 60 mm	Raspberry Pi 5, ESP32-S3, BNO085 IMU	Shortest camera cables through the axle; IMU close to the rotation axis
Deck B, 125 mm	Dual 20 A motor driver, relays K1 and K2, kill receiver, 25 A fuse, 5 V and 6 V buck converters	Power parts grouped, short high-current runs to the motors
Outboard of the side plates, 100 mm	Two drive gearmotors and belts	In line with the hub pulleys
Sled, 185 - 266 mm	Carriage, 6S2P battery, 3.98 kg steel ballast	Heaviest parts lowest: they set the centre of gravity and do the steering

The camera ribbon cables run through the hollow axle with a service loop, because the axle turns up to ±60° against the pendulum. The polycarbonate shell is transparent to 2.4 GHz, so the kill receiver and Wi-Fi need no external antenna.

Mass budget and centre-of-gravity placement

RoToRBot weighs 14.0 kg, and its centre of gravity sits 116 mm below the sphere centre, on the vertical through the contact point. The pendulum carries 70 % of the mass (9.75 kg), and half of that is the 4.91 kg sled at the very bottom.



Group	Mass (kg)	Depth of its centre below the sphere centre (mm)	Mass × depth (kg·m)
Shell: hemispheres, flange, band, pole hubs	4.24	0	0
Fixed axle and camera pods	0.59	0	0
Pendulum frame: side plates, decks	0.96	86	0.083
Drive: 2 gearmotors, pulleys, belts	1.60	94	0.150
Electronics and levelling servo	0.51	81	0.041
Wiring and fasteners (incl. 0.30 kg contingency)	0.60	100	0.060
Steering drive: stepper, rails, lead screw	0.58	170	0.099
Steering sled: carriage, battery, ballast	4.91	243	1.192
Total	**13.99**	**116 (centre of gravity)**	**1.625**

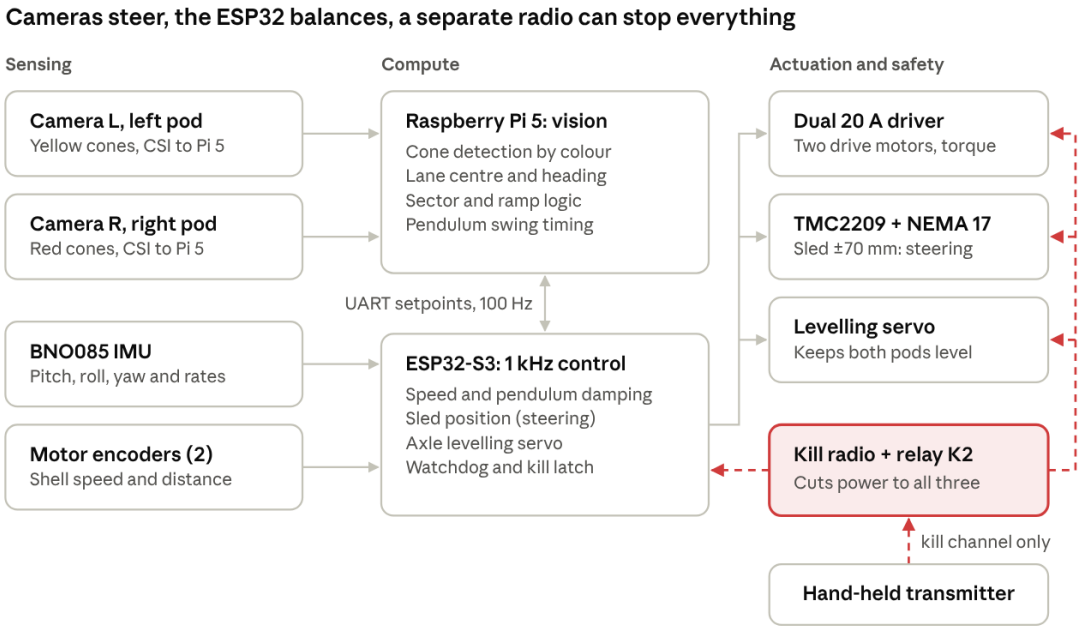
- **Drive authority:** the total mass × depth, 1.625 kg·m, is the pendulum moment that pushes the ball. It sets the climbing limit in the sizing section.
- **Self-righting:** tipped 10° by a bump or an obstacle, the robot feels a 2.8 N·m restoring moment and settles upright.
- **Lateral trim:** the stepper on one side puts the centre of gravity 2.4 mm off-axis. The sled's neutral position is set 6.8 mm the other way to cancel it.

- **Clearance:** at full sled travel the ballast corner is 12.7 mm from the shell; the closest fixed part, a motor bracket, clears it by 21.9 mm.
- **Light shell:** the shell is 30 % of the mass (rotational inertia 0.29 kg·m²), which keeps the pendulum moment high for the total weight.

Masses are estimates from material volumes and typical part weights, with 0.30 kg of contingency. They will be replaced by weighed values on the prototype.

System architecture schematic

Two computers split the work: the Raspberry Pi 5 decides where to go from the cameras, and the ESP32-S3 makes the ball do it.



System architecture · sensing, compute, actuation and the independent kill path

The Pi sends speed and curvature targets to the ESP32 at 100 Hz, and the ESP32 closes every motor loop at 1 kHz. The kill radio and relay K2 sit outside both computers; the ESP32 only reads their state (dashed red line).

Control system

RoToRBot is driven by a two-part torque vector: pitch torque from the two drive motors and roll torque from the steering sled. Each actuator has its own driver channel, so the controller sets the vector directly. This is the linear-combination control we first built for our earlier spherical robot, carried over to a larger, autonomous ball.

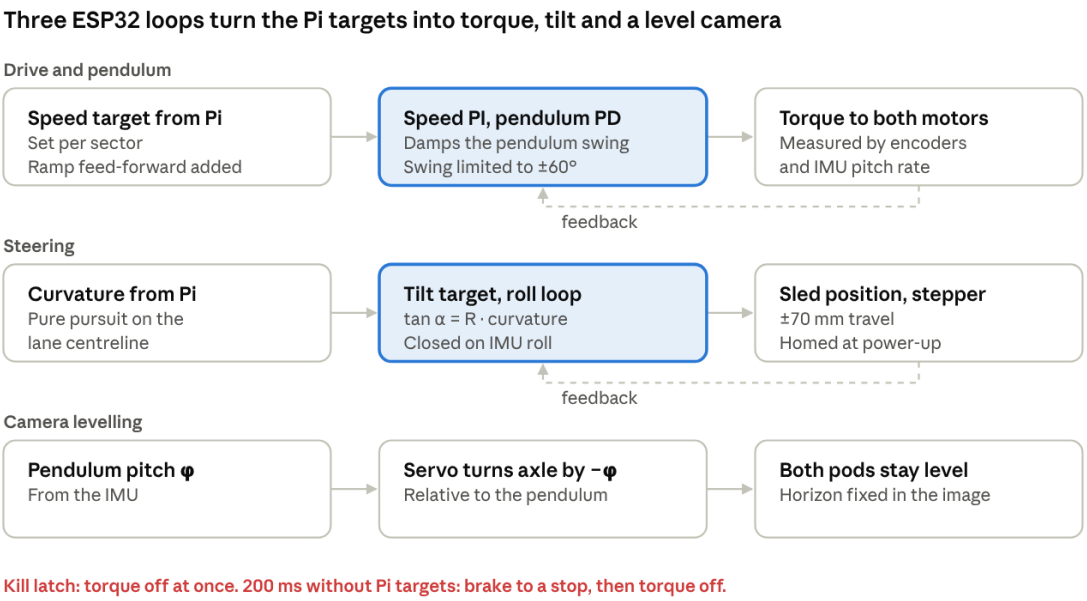
(4) Torque vector as a linear combination of the actuators, and its inverse

$$\tau_{pitch} = 3\,\eta\,(\tau_L + \tau_R), \quad \tau_{roll} = m_{sled}\,g\,d \qquad \Rightarrow \qquad \tau_L = \tau_R = \frac{\tau_{pitch}}{6\,\eta}, \quad d = \frac{\tau_{roll}}{m_{sled}\,g}$$

(5) H-bridge PWM duty for each motor, with back-EMF and bus-voltage compensation; ground speed

$$u_i = \frac{R_a\,\tau_i/k_t + k_e\,\omega_i}{V_{bus}}, \quad |u_i| \le 1 \qquad\qquad v_{ground} = R\,(\omega_{rel} + \dot{\varphi})$$

- **Actuators and drivers:** each drive gearmotor has its own channel of the dual 20 A H-bridge, switched at 20 kHz and current-limited to 10 A. The sled stepper runs on a TMC2209, and the levelling servo on a standard servo pulse.
- **Linear combination:** the controller asks for a pitch torque (to drive or brake) and a roll torque (to turn), and equation (4) maps them onto the actuators. Pitch and roll act about different axes, so changing one does not disturb the other.
- **Two motors, one shell:** the motors turn the same shell from opposite poles, so they split the pitch torque equally. A trim on the two channel currents stops one motor fighting the other.
- **PWM law:** equation (5) turns each torque request into a duty cycle. It adds the motor's back-EMF and divides by the measured bus voltage, so the same request gives the same torque on a full or a tired battery.
- **State estimation:** ground speed adds the encoder speed of the shell relative to the pendulum to the pendulum's own pitch rate from the IMU. Distance along the track is integrated from it, so the robot knows which sector it is in.
- **Timing:** every loop runs in a 1 kHz scheduler on the ESP32-S3, and the Pi refreshes the targets at 100 Hz.



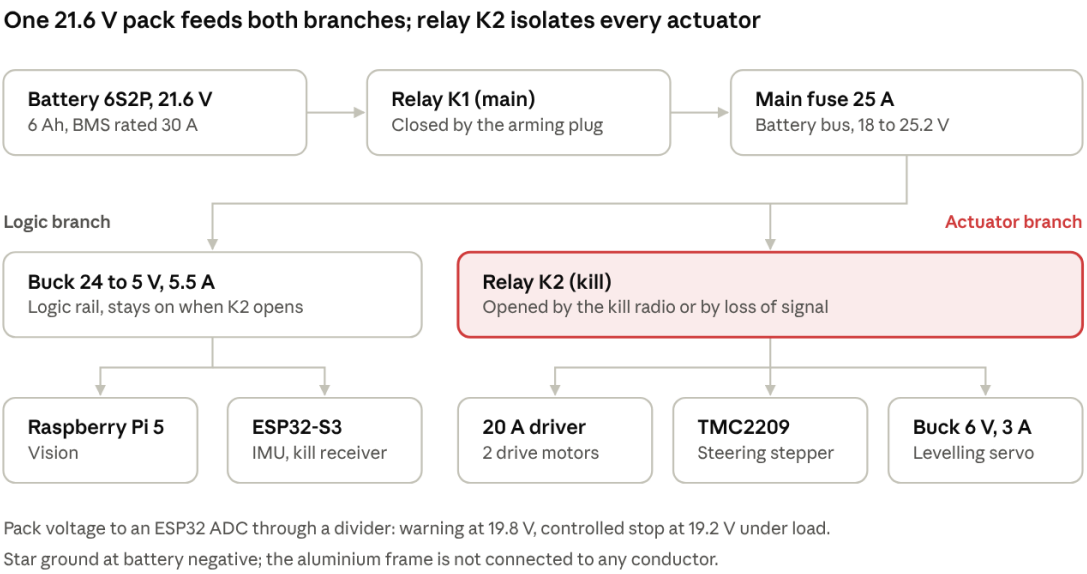
Control loops on the ESP32-S3, all scheduled at 1 kHz

The drive loop is the one that keeps the ball stable: the speed loop asks for a pendulum angle, and a fast PD loop on that angle damps the 1.6 s rocking mode. On ramps, the torque feed-forward $m \cdot g \cdot R \cdot \sin\theta$ is added before the loop has to react.

Power system

Competition power system (baseline)

A single economic (SIMILAR TO 62SP) custom made Li-ion pack (21.6 V nominal, 25.2 V full, 6 Ah, 130 Wh) powers everything. After the main relay and fuse it splits into an always-on logic branch and an actuator branch behind kill relay K2.



Power architecture · logic branch and kill-switched actuator branch

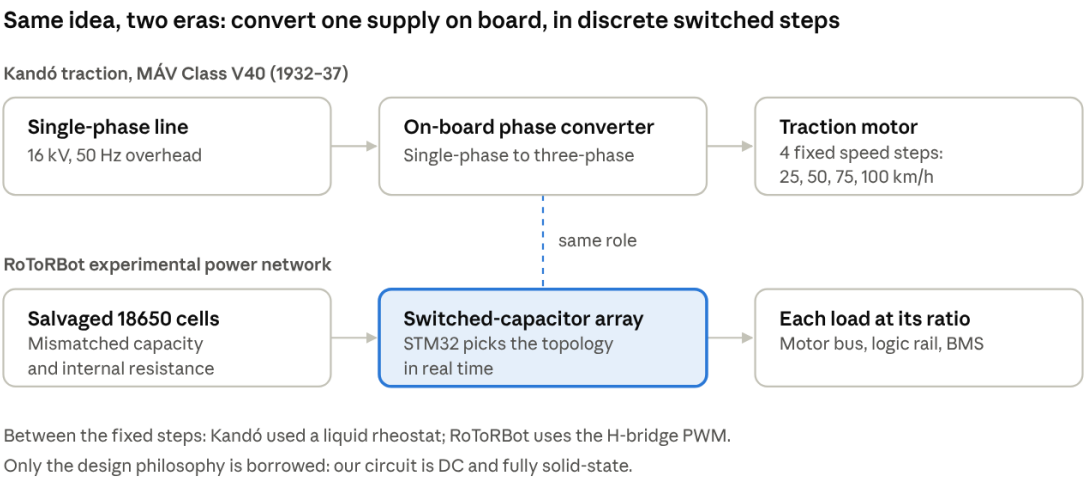
The 5 V logic rail stays up when K2 opens, so the ESP32 can latch the fault and the Pi keeps its log. Every actuator, the levelling servo included, sits downstream of K2. Peak draw is about 25 A (two motors limited to 10 A each, plus stepper and logic), within the 30 A rating of the 2P high-drain cells and BMS.

For electrical isolation, all conductors are insulated, connectors are keyed XT60s, and the aluminium frame is bonded to nothing. Before each run we check frame-to-battery resistance with a multimeter and expect an open circuit.

Experimental: Kandó-inspired adaptive capacitor network (our long-term goal)

Our own power concept, first proposed for our earlier spherical robot, replaces fixed converters with a switched-capacitor network that an STM32 reconfigures in real time. It borrows only the design philosophy of Kálmán Kandó's railway traction system.

Kandó's MÁV Class V40 locomotives (1932- 37) took a single-phase 16 kV, 50 Hz line and converted it on board to three-phase. The motor ran at four fixed speeds, with a liquid rheostat for the speeds in between. Our network does the same job for a DC robot: one supply, converted on board, in discrete switched steps.



Kandó traction and the RoToRBot power network , V40 facts from Wikipedia

- **Switched-capacitor array:** MOSFETs switch banks of capacitors between series and parallel arrangements. Each arrangement transfers energy at a fixed voltage ratio, acting like a transformer with no magnetic core.
- **STM32 controller:** it reads every cell voltage, the bus voltages and the load currents, and changes the topology in real time.
- **What it balances:** energy flow between the sensor and communication loads, the PWM motor drivers, the BMS, and the uneven discharge curves of low-grade cells salvaged from broken electronics.
- **Safety boundary:** the network sits after relay K1 and before relay K2, so the kill path stays hardware-only. A bypass relay returns the robot to the baseline system at any time.
- **Status:** bench prototype. The competition robot runs on the baseline system above, and the module is fitted only after it passes bench tests. Its estimated mass, under 0.25 kg, fits inside the 0.30 kg contingency.

Improvements this power system targets

The adaptive network is aimed at seven improvements over a fixed battery-plus-converter system. Each has a target we will measure on the bench and report in Phase 2.

Improvement	How the network delivers it	Target to measure
Steady motor-bus voltage	Steps the capacitor ratio up as the pack sags, so torque per PWM step stays constant and the sizing model holds for the whole run	Bus within ±5 % from full to empty; the bare pack falls 29 % (25.2 to 18.0 V)
Lower battery peak current	Charged capacitors supply the short bursts of acceleration and ramp climbing	Peak battery current 30 % lower on the 0.8 ft ramp

Improvement	How the network delivers it	Target to measure
More energy from salvaged cells	Moves charge from strong cells to weak ones instead of bleeding the strong ones as heat	15 - 25 % more usable capacity than passive balancing
No logic brownouts	The logic rail is fed from its own capacitor stage, isolated from motor dips	No Pi or ESP32 reset in a stalled-motor test
Braking energy recovered	Braking before the pendulum and on ramp descents charges the array	Store the 19 J of one stop from 1.5 m/s
Lighter, quieter conversion	No inductors; switching at fixed ratios	Less magnetic noise near the IMU and radios
Battery health data	The STM32 logs every cell and the ratio it chose	Per-cell discharge curves for choosing cells

Wireless emergency kill switch

The kill switch is a separate 2.4 GHz radio that cuts power to every actuator through relay K2. It does not depend on the Raspberry Pi or the ESP32.

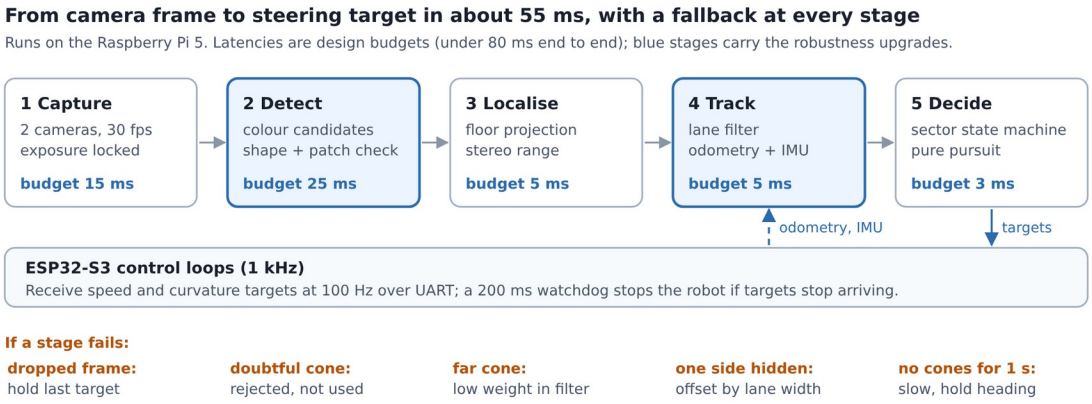
1. A hand-held 2.4 GHz RC transmitter talks only to its own receiver on deck B. The link carries the kill channel and nothing else, so the run stays fully autonomous.
2. The receiver's kill channel drives relay K2 through a pulse-to-switch circuit. K2 sits in the 24 V feed to both motor drivers, the steering stepper driver and the 6 V servo supply, so opening it stops all motor actuation at once.
3. K2 is held closed only while valid RUN pulses arrive. A thrown switch, a transmitter switched off, a lost signal or an unpowered receiver all open it. The receiver's own failsafe output is also set to KILL.
4. As a second layer, the ESP32 reads the kill channel and K2's state. It zeroes every PWM output, disables the drivers and latches a fault, so nothing restarts when K2 closes again.
5. Pulling the arming plug from the left pod opens main relay K1 and removes all power.

Event	Relay K2	ESP32	Result
Kill switch thrown	Opens	Zero PWM, fault latched	All actuators stop
Transmitter off or out of range	Opens (failsafe)	Zero PWM, fault latched	All actuators stop
Receiver loses power	Opens (coil unpowered)	Zero PWM, fault latched	All actuators stop
Pi stops sending commands	Stays closed	200 ms watchdog: brakes, then zero PWM	Robot stops in place
Arming plug pulled	K1 opens	Off	All power off

Start sequence: insert the arming plug, step back, set the transmitter to RUN; the robot starts after a 3 s countdown shown on a status LED visible through the shell. At Day 1 inspection we will show each kill path with the shell spinning on a stand.

Autonomy: vision pipeline

Autonomy runs as a five-stage pipeline on the Pi 5, from camera frame to steering target in about 55 ms. Every stage has a fallback, so a bad frame or a hidden cone slows the robot instead of sending it off course.



1. **Capture:** both Camera Module 3 Wide units stream to the Pi 5 over CSI, scaled to 768 × 432 and processed at 30 fps per camera. Exposure and white balance are locked after the Sector 1 calibration, so colours do not drift with the light. Every frame is time-stamped against the IMU.
2. **Detect:** HSV thresholds pick out cone red (two hue bands) and cone yellow as candidates. Each candidate must pass a shape check: a triangular outline, a base width that matches its distance, and a base on the floor. A small classifier, trained on our own practice-day images, then checks only these candidate patches and rejects cardboard, thermocol or tape of a similar colour.
3. **Localise:** each cone base is projected onto the floor from the camera height (320 mm), the IMU pitch and roll, and the levelling-servo angle. Cones seen by both cameras also get stereo range on the 0.73 m baseline. Every position carries an uncertainty that grows with distance.
4. **Track:** a Kalman filter follows four lane states: centre offset, heading error, curvature and lane width. Encoder and IMU odometry predict it at 100 Hz and each cone corrects it; a cone that does not fit the lane is rejected as an outlier. If only one colour is visible, the centreline is set from that side using the lane width learned in Sector 1.
5. **Decide:** pure pursuit to a look-ahead point on the centreline gives the curvature target. The look-ahead grows with speed, from 1.0 m when slow to 1.5 m at the 1.5 m/s cruise. The sector state machine below sets the speed and behaviour.
6. **Pendulum:** in Sector 4 the robot stops 1 m short, and both cameras follow the bob by frame differencing in a window above the track. Stereo gives its position across the lane, and a sine fit over two swings gives the period and phase for the go/no-go rule.

Sector state machine

Distance alone never changes the sector, because the shell slips on ramps and when pushing obstacles. Each change needs the robot inside a distance window and a confirming cue from its sensors. The windows overlap to absorb ±3 m of odometry drift.

State	Enters when	Behaviour
Gate Runway	Start of the run	Calibrate colours, exposure and lane width on the first gates; accelerate in steps to 1.5 m/s
Step-Up Ramps	12 - 40 m travelled and IMU pitch rises above 4°	Torque feed-forward; hold heading on the IMU while the cameras look up the slope; 2.0 m/s run-up before the 0.8 ft ramp
Wrecking Zone	32 - 60 m travelled and obstacles in view, or repeated sudden slowdowns	Steer on cones only; hold 1.2 m/s; add torque when the encoders show it slowing
Pendulum Gauntlet	52 - 75 m travelled and a swinging object detected ahead	Stop 1 m short, measure period and phase, dash when the window is safe
Finish	75 m passed and the pendulum cleared	Roll 1 m past the line, then brake to a stop

Failure behaviours

Each foreseeable failure has a detection rule and a set response, so the robot degrades gracefully instead of guessing.

Situation	How it is detected	Response
One side of cones hidden	No cones of one colour for 0.5 s	One-sided lane: offset from the visible side by half the learned lane width
No cones at all	Nothing detected for 1 s	Slow to 0.5 m/s and hold the IMU heading; after 3 m, stop and creep at 0.3 m/s until cones return
Cone too close	Predicted gap to the ball's edge under 0.25 m	Sled to full travel away from it; speed cut by half
Stall on a ramp or obstacle	Encoder speed near zero at high torque for 1 s	Back off 1 m and retry with a longer run-up or full torque
Pendulum timing unsure	Fitted period varies by more than 10 % between swings	Wait for another swing; never dash on an uncertain window
Pi lags or freezes	Targets older than 150 ms	ESP32 holds the last curvature and slows; the 200 ms watchdog then stops the robot

Testing plan and targets

Every practice run is logged (both cameras, IMU, encoders and commands), so the pipeline can be replayed offline after each change. These are the targets we will measure and report in Phase 2.

Metric	Target	How we measure it
Cone detection	≥ 95 % of cones found and ≥ 98 % of detections correct, including Sector 3 clutter	Hand-labelled frames from practice-day video
Lane-centre error	Under 10 cm RMS	Against a taped centreline on our test track
Frame-to-command latency	Under 80 ms end to end	An LED flashed in view; time to the PWM change on an oscilloscope
Sector changes	20 of 20 correct with ±3 m of odometry error injected	Replay of logged runs
Pendulum go/no-go	No unsafe GO in 50 trials	Mock pendulum on the test track
Full track	A clean run with no cone contact	Full-length mock track before the finals

Before hardware runs, the controller and the pipeline are tuned on a simulated track with cones, ramps, obstacles and a pendulum, using the ball's dynamics model from the sizing section. The same simulation supports the Round 2 virtual audit.

Sector-by-sector run strategy

RoToRBot runs fast where the track is open and slows only where a sector demands it; the target is about 75 s for the 75 m with no cone contact.

Sector	Track	What RoToRBot does	Speed target	Fallback
1. Gate Runway, 0 - 15 m	Straight run, chevron cone gates	Tunes colour thresholds and camera pitch on the first gates and measures lane width, then raises speed in steps the pendulum can hold	Up to 1.5 m/s (about 12 s)	Poor lighting: keep the thresholds from practice runs
2. Step-Up Ramps, 15 - 35 m	Ramps of 0.2, 0.4 and 0.8 ft through cone gates	Detects each ramp from IMU pitch and the ramp edge in the image; adds torque feed-forward $m \cdot g \cdot R \cdot \sin\theta$; holds heading on the IMU while the cameras look up the slope; 2.0 m/s run-up before the 0.8 ft ramp	1.0 m/s on ramps (about 22 s)	Stall: roll back 1 m and retry with a longer run-up
3. Wrecking	Foam, thermocol	Steers on cone colours only, so obstacles never pull the heading; carries 1.2 m/s (17	1.2 m/s (about	An obstacle pushed toward

Sector	Track	What RoToRBot does	Speed target	Fallback
Zone, 35 - 55 m	and cardboard obstacles	kg·m/s of momentum) and adds torque when the encoders show it slowing	17 s)	a cone: shift the lane centre away from that cone line
4. Pendulum Gauntlet, 55 - 75 m	Pendulum swinging across the track	Stops 1 m short of the swing zone, measures period and phase over two swings with both cameras, then dashes as the bob clears the lane	Hold, then 1.5 - 2.0 m/s (about 23 s with the wait)	Bob mistimed: brake and wait for the next swing

For the pendulum dash, RoToRBot goes only when (d + b + D) / v < T/2 - 0.3 s. Here d is the 1 m stand-off, b the bob depth, D the 0.64 m ball, v the dash speed and T the measured period.

Preliminary sizing: torque, ramp limit, speed and turning

Climbing a 15° ramp needs 12.0 N·m at the shell. Two motors at 3:1 deliver it with a 49° pendulum swing, inside the 60° limit. Phase 2 will extend these static numbers into the full simulation.

(1) Pendulum swing needed on a ramp of angle θ

$$\sin\varphi = \frac{m R (\sin\theta + C_{rr}\cos\theta)}{\sum m_i l_i}$$

(2) Shell torque, and torque per motor through the 3:1 belts

$$\tau_{shell} = m g R (\sin\theta + C_{rr}\cos\theta) \qquad \tau_{motor} = \frac{\tau_{peak}}{2 \cdot 3 \cdot \eta_{belt}}$$

(3) Steering tilt from the sled shift d, and turn radius (quasi-static)

$$\tan\alpha = \frac{m_{sled}\, d}{m\, h_{cg}} \qquad r \approx \frac{R}{\tan\alpha}$$

Symbols: m total mass, R rolling radius, $\Sigma m_i l_i$ pendulum moment (mass × depth below the centre), C_rr rolling resistance, η_belt belt efficiency, d sled shift, h_cg depth of the centre of gravity.

Quantity	Value	Basis
Total mass m; rolling radius R	13.99 kg; 0.320 m	Mass budget; 640 mm OD
Pendulum moment Σ m·l	1.625 kg·m	Mass budget
Swing needed on a 10° / 15° / 17° ramp	31° / 49° / 58°	Rolling resistance C_rr = 0.015
Steepest ramp at the 60° swing limit (90° absolute)	18.3° (21.3°)	Same model
Shell torque on a 15° ramp	12.0 N·m	τ_shell formula
Design peak shell torque	17.5 N·m	17° ramp plus 30 % for acceleration and losses
Torque per motor	3.1 N·m at 134 rpm	2 motors, 3:1 belt, 95 % belt efficiency
Climbing power, 15° at 1.5 m/s	59 W	Both motors together
Shell speed at 1.5 m/s cruise	44.8 rpm	v / R
Height gained on stored energy alone	138 mm from 1.5 m/s; 245 mm from 2.0 m/s	Kinetic energy including shell rotation
Grip needed on a 15° ramp	μ ≥ 0.27	tan θ; rubber band on board
Steering tilt and turn radius	11.9°, about 1.5 m	Sled ±70 mm
Rocking period, motors holding	1.6 s	Small-angle rocking on a flat floor

This sets the motor requirement: 24 V, 45 mm planetary gearmotor with a quadrature encoder, at least 3.1 N·m continuous at 140 rpm and about 190 rpm no-load. The problem statement gives ramp heights (0.2, 0.4 and 0.8 ft) but not their lengths. If the 0.8 ft ramp is steeper than 18°, a 2.0 m/s run-up stores enough energy to lift the ball the full 245 mm before the drive contributes anything.

Bill of materials (ESTIMATED SPECS MAY VARY WRT SUBSTITUTION AND COST EFFICIENCY)

RoToRBot uses 24 line items plus one optional experimental module, all commonly available; item numbers match the internal layout figure.

#	Part	Qty	Specification
1	Shell hemispheres	2	Thermoformed clear polycarbonate, 640 mm OD, 3 mm sheet (about 2 mm after forming)

#	Part	Qty	Specification
2	Traction band	1	Rubber strip 60 × 2 mm, about 2.0 m, bonded over the seam
3	Equator flange ring	1	Internal ring 16 mm wide, M4 bolts
4	Pole hubs	2	Aluminium flange with 6805-2RS bearing (25 × 37 × 7 mm)
5	Hub pulleys	2	54T HTD-5M, 15 mm wide, bolted to the hubs
6	Fixed axle	1	6061 aluminium tube 25 × 2.5 mm, 740 mm long
7	Camera pods	2	Housing, lens window, passive socket; Raspberry Pi Camera Module 3 Wide in each
8	Levelling servo and axle gear	1	Metal-gear digital servo, about 20 kgf·cm
9	Pendulum frame	1 set	Two 4 mm 6061 side plates, decks A and B, two axle bearings
10	Drive gearmotors	2	24 V, 45 mm planetary, quadrature encoder, ≥ 3.1 N·m at 140 rpm, about 190 rpm no-load
11	Motor pulleys and belts	2	18T HTD-5M pulley; HTD-5M belt, 15 mm wide, for 100 mm centres
12	Steering drive	1	NEMA 17 stepper, T8 lead screw (8 mm lead), two 8 mm rails
13	Sled carriage	1	Four LM8UU bearings, T8 nut, hanger plate
14	Battery	1	6S2P Li-ion, high-drain 18650 cells (≥ 15 A each), 21.6 V, 6 Ah, 6S BMS rated ≥ 30 A
15	Ballast	1	Steel plate 130 × 130 × 30 mm, 3.98 kg, adjustable
16	Vision computer	1	Raspberry Pi 5, 8 GB, active cooler
17	Real-time controller	1	ESP32-S3 development board
18	IMU	1	BNO085, 9-axis with on-chip fusion
19	Motor driver	1	Dual-channel 20 A brushed DC driver, 24 V
20	Stepper driver	1	TMC2209
21	Buck converters	2	24 to 5 V, 5.5 A (Pi 5 and logic); 24 to 6 V, 3 A (servo)
22	Relays	2	K1 main and K2 kill: 24 V coil, 40 A
23	Kill radio	1 set	2.4 GHz RC transmitter and receiver with failsafe

#	Part	Qty	Specification
			output, pulse-to-switch board
24	Protection and wiring	1 set	25 A fuse, XT60 connectors, arming plug, silicone wire, status LED
25	Experimental power module (optional)	1	STM32 board, MOSFET switch array, capacitor banks, bypass relay; bench prototype

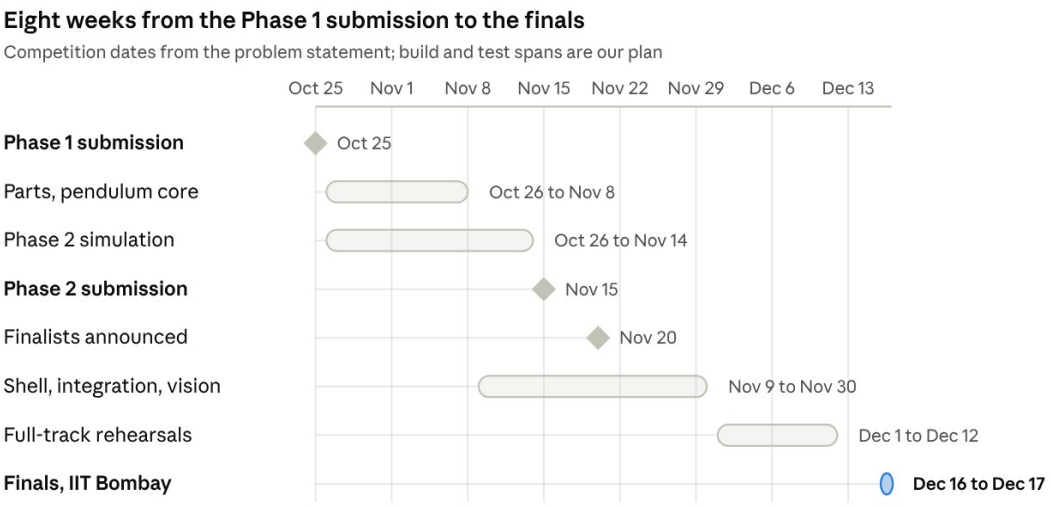
Risks, open questions and plan to the finals

The largest open risk is the ramp angle, which the problem statement does not give; the design already has a momentum fallback for it.

Risk	Effect	Mitigation
0.8 ft ramp steeper than 18°	Pendulum cannot hold a steady climb	2.0 m/s run-up lifts 245 mm on stored energy; ballast can grow by 1 kg; angle asked in the Doubt section
Pendulum rocking at speed	Camera shake, lane lost	PD damping on pendulum angle, levelled pods, speed capped per sector
Arena lighting differs from practice	Colour thresholds drift	Re-tune on the Sector 1 chevron gates; fixed exposure; stereo cross-check
Cardboard hits a camera pod	Pod knocked or damaged	Pods only 83 mm out, rounded, with a guard ring; lane bias away from obstacles
Turning radius about 1.5 m	Late correction clips a gate cone	1.5 m look-ahead starts corrections early; slow down before tight gates
Thermoformed shell thinner or out of round	Bumpy roll, weaker shell	Measure wall thickness on delivery; band covers the seam; spare hemisphere
Experimental power module not proven in time	None on the run	The robot runs on the baseline power system; the module stays on the bench until it passes its tests

Questions we are raising in the Doubt section:

- Lengths or angles of the 0.2, 0.4 and 0.8 ft ramps
- Lane width between the red and yellow cones, and gate spacing
- Pendulum bob size, swing period and height above the floor
- Floor surface and lighting in the arena
- Whether the run may be started from the kill-switch transmitter



Plan to the finals · competition dates from the problem statement, build spans planned

The bench test of the pendulum core comes before the shell is closed, so motor sizing and the damping loop are proven early and fed into the Phase 2 report.